

CUD MANAGEMENT SYSTEM

Product Requirements Document (Production Ready)

Organization: CASFETA – Christ Ambassadors Students Fellowship

Chapter: CUD Chapter

Your Role

You are an expert team consisting of:

- Senior Software Architect
- Senior Next.js Full Stack Developer
- Senior UI/UX Designer
- Senior PostgreSQL Database Engineer
- Senior Security Engineer
- Senior DevOps Engineer

Your task is to build a **complete production-ready CUD Management System**.

This is **not a prototype** and **not a demo**.

Do not generate placeholder code.

Do not leave TODO comments.

Every feature must be fully functional.

Write clean, scalable, secure, maintainable, and well-documented code following industry best practices.

Purpose

Develop a centralized web-based management system for **CASFETA CUD Chapter** that allows leaders to manage members, communication, ministries, attendance, skills, and documents from one secure platform.

Problem Statement

The current fellowship management process is manual.

Leaders struggle to:

- Register members accurately.
- Search members based on skills or course.
- Communicate efficiently.
- Keep attendance records.
- Store important documents safely.
- Organize ministries and leadership information.
- Generate reports.

The new system should solve all of these challenges.

Goals

The system should:

- Register and manage all fellowship members.
 - Search members by skills in under five seconds.
 - Send SMS announcements to selected groups or all members.
 - Store important documents securely.
 - Manage ministries and leadership.
 - Record attendance.
 - Generate professional reports.
 - Support future expansion without major redesign.
-

User Roles

Super Admin

Examples:

- Chairman
- Vice Chairman
- Secretary
- Treasurer
- Pastor
- Teacher
- ICT Administrator

Permissions:

- Full access to every module.
 - Manage users.
 - Approve member skills.
 - Approve registrations if needed.
 - Create announcements.
 - Manage ministries.
 - Upload documents.
 - View reports.
 - Configure system settings.
 - Manage roles and permissions.
-

Admin

Examples:

Ministry Leaders

Permissions:

- Manage members within their ministry.
 - Record attendance.
 - Approve ministry requests.
 - Upload ministry documents.
 - Create announcements.
 - View ministry reports.
-

Member

Permissions:

- Register.
 - Update profile.
 - Select skills.
 - Join ministries.
 - Download documents.
 - View announcements.
 - Comment on announcements.
 - View personal attendance.
-

Core Features

1. Member Registration

Include:

- Full Name
 - Registration Number
 - Phone Number
 - Email Address
 - Gender
 - Faculty
 - Department
 - Course
 - Year of Study
 - Hostel
 - Home Region
 - Emergency Contact
 - Profile Photo
 - Biography
 - Multiple Skills
-

2. Skills Management

Workflow:

Student selects skills



Status becomes Pending



Leader reviews



Approve or Reject



Approved skills become Active

Every approval should be recorded in audit logs.

3. Member Directory

Allow searching by:

- Name
- Registration Number
- Phone Number
- Faculty
- Department
- Course
- Year
- Hostel
- Ministry
- Skill
- Gender

Searching should be extremely fast.

4. Communication

Features:

- Announcements
- Comments

- SMS Broadcast using Africa's Talking API
- Notification system

Allow sending SMS to:

- All members
- Ministry members
- Course
- Faculty
- Hostel
- Custom selected members

5. Document Management

Support:

- PDF
- Word
- Excel
- Images

Categories:

- Constitution
- Meeting Minutes
- Timetables
- Letters
- Financial Documents
- Schedules

Allow upload, download, preview, search, and permission-based access.

6. Ministry Management

Each ministry should have:

- Leader
- Assistant Leader
- Members
- Documents

- Attendance
 - Announcements
-

7. Attendance

Support attendance for:

- Sunday Fellowship
- Bible Study
- Prayer Meetings
- Conferences
- Evangelism

Generate attendance reports by:

- Date
 - Month
 - Semester
 - Ministry
 - Member
-

8. Events

Allow leaders to:

- Create events
 - Update events
 - Delete events
 - Register participants
 - View attendees
-

9. Reports

Generate reports for:

- Members
- Ministries
- Skills

- Attendance
- Events

Allow export to:

- PDF
 - Excel
-

Technical Requirements

Frontend

- Next.js 15 App Router
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

Backend

- Next.js Route Handlers
- Server Actions where appropriate

Database

- Supabase PostgreSQL

Authentication

- Supabase Auth

Storage

- Supabase Storage

SMS

- Africa's Talking API

Deployment

- Vercel

Security Requirements

Implement:

- Role-Based Access Control (RBAC)
- Protected Routes
- Secure Authentication
- Server-side validation
- Client-side validation
- Input sanitization
- Audit Logs
- Secure file uploads
- Environment variables for secrets

Never expose API keys.

Performance Requirements

- Responsive on desktop, tablet, and mobile.
 - Fast page loading.
 - Fast searching.
 - Optimized database queries.
 - Pagination for large datasets.
 - Lazy loading where appropriate.
-

UI/UX Requirements

Create a clean, modern, professional dashboard with:

- Beautiful cards
- Responsive sidebar
- Top navigation
- Search bar
- Data tables

- Charts
- Statistics cards
- Light and Dark Mode
- Toast notifications
- Loading skeletons
- Empty states
- Error pages
- Confirmation dialogs

The design should feel polished, premium, and intuitive.

Database

Design a normalized PostgreSQL database.

Create:

- ER Diagram
- Database schema
- Relationships
- Constraints
- Indexes
- Foreign Keys

Generate SQL migration files.

API

Build complete REST API endpoints for every module.

Include:

- Authentication
- Members
- Skills
- Ministries
- Attendance
- Documents

- Events
- Reports
- Notifications

Implement proper validation and error handling.

Deliverables

Generate:

1. Complete project folder structure.
2. Database schema.
3. SQL migrations.
4. API routes.
5. Server Actions.
6. Authentication system.
7. Frontend pages.
8. Dashboard.
9. Admin panel.
10. Member portal.
11. Responsive UI.
12. Production-ready code.
13. Environment variable template (.env.example).
14. Installation guide.
15. Deployment guide.
16. Sample seed data.
17. Comprehensive README.

The final output should be a fully functional, secure, scalable, production-ready application with no missing functionality and code that follows modern software engineering best practices.